

a. Name of two GenAI programs: **ChatGPT, Claude code**

b. How you accessed them (e.g., Google query, CoPilot on my computer): **My openAI account web interface and anthropic account web interface**

c. Prompt used. (You must use the same prompt for both GenAIs.)

prompt used:

Can you generate C code to implement the following requirements:

The program will prioritize emails for a busy company CEO.

- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules.

Here is a sets file format:

Command Description

EMAIL == The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows: • Boss – read first • Subordinate – read next • Peer – read next • ImportantPerson – read next • OtherPerson – read last If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint. EMAIL is followed by space. The rest of the fields are delimited. Assume is one of the five strings listed above. Assume is a string which may contain spaces, but not commas Assume is in the format: MM-DD-YYYY

NEXT == Next email for the CEO to read. Display the information on the terminal in the following format:
Sender: Subject: Date:

READ == CEO has read email and dealt with it

COUNT == Display current count of unread emails

Importantly: The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

The output of this file must be:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read

d. Generated code from both GenAIs.

Chat GPT

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#define MAX_EMAILS 1000
```

```
#define MAX_SUBJECT 500
```

```
#define MAX_SENDER 50
```

```
#define MAX_DATE 11
```

```
typedef struct {
```

```
    char sender[MAX_SENDER];
```

```
    char subject[MAX_SUBJECT];
```

```
    char date[MAX_DATE];  
    int priority;  
} Email;
```

```
typedef struct {  
    Email emails[MAX_EMAILS];  
    int size;  
} MaxHeap;
```

/* get the priority value for each sender category */

```
int getPriority(char sender[]) {  
    if (strcmp(sender, "Boss") == 0)  
        return 5;  
    else if (strcmp(sender, "Subordinate") == 0)  
        return 4;  
    else if (strcmp(sender, "Peer") == 0)  
        return 3;  
    else if (strcmp(sender, "ImportantPerson") == 0)  
        return 2;  
    else  
        return 1;  
}
```

/* convert a date into a number that can be compared */

```
int dateValue(char date[]) {  
    int month, day, year;  
  
    sscanf(date, "%d-%d-%d", &month, &day, &year);  
  
    return year * 10000 + month * 100 + day;  
}
```

/* determine whether one email has higher priority than another */

```
int higherPriority(Email a, Email b) {  
    if (a.priority > b.priority)
```

```

        return 1;

    if (a.priority < b.priority)
        return 0;

    return dateValue(a.date) > dateValue(b.date);
}

/* swap two emails */
void swap(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

/* move an email upward through the heap */
void heapifyUp(MaxHeap *heap, int index) {
    int parent;

    while (index > 0) {
        parent = (index - 1) / 2;

        if (higherPriority(heap->emails[index], heap->emails[parent])) {
            swap(&heap->emails[index], &heap->emails[parent]);
            index = parent;
        } else {
            break;
        }
    }
}

/* move an email downward through the heap */
void heapifyDown(MaxHeap *heap, int index) {
    int left;
    int right;

```

```

int largest;

while (1) {
    left = 2 * index + 1;
    right = 2 * index + 2;
    largest = index;

    if (left < heap->size &&
        higherPriority(heap->emails[left], heap->emails[largest])) {
        largest = left;
    }

    if (right < heap->size &&
        higherPriority(heap->emails[right], heap->emails[largest])) {
        largest = right;
    }

    if (largest != index) {
        swap(&heap->emails[index], &heap->emails[largest]);
        index = largest;
    } else {
        break;
    }
}

```

/* add an email to the MaxHeap */

```

void insertEmail(MaxHeap *heap, Email email) {
    if (heap->size >= MAX_EMAILS) {
        return;
    }

```

```

    heap->emails[heap->size] = email;

```

```

    heapifyUp(heap, heap->size);

```

```

    heap->size++;

```

```
}
```

```
/* remove and return the highest priority email */
```

```
Email removeMax(MaxHeap *heap) {
```

```
    Email result;
```

```
    if (heap->size == 0) {
```

```
        result.sender[0] = '\0';
```

```
        result.subject[0] = '\0';
```

```
        result.date[0] = '\0';
```

```
        result.priority = 0;
```

```
        return result;
```

```
    }
```

```
    result = heap->emails[0];
```

```
    heap->size--;
```

```
    if (heap->size > 0) {
```

```
        heap->emails[0] = heap->emails[heap->size];
```

```
        heapifyDown(heap, 0);
```

```
    }
```

```
    return result;
```

```
}
```

```
/* look at the highest priority email without removing it */
```

```
Email peekMax(MaxHeap *heap) {
```

```
    Email result;
```

```
    if (heap->size == 0) {
```

```
        result.sender[0] = '\0';
```

```
        result.subject[0] = '\0';
```

```
        result.date[0] = '\0';
```

```
        result.priority = 0;
```

```
        return result;
```

```

    }

    return heap->emails[0];
}

/* display the next email */
void displayNext(MaxHeap *heap) {
    Email email;

    if (heap->size == 0) {
        return;
    }

    email = peekMax(heap);

    printf("Next email:\n\n");
    printf("Sender: %s\n", email.sender);
    printf("Subject: %s\n", email.subject);
    printf("Date: %s\n\n", email.date);
}

/* read and remove the highest priority email */
void readEmail(MaxHeap *heap) {
    if (heap->size > 0) {
        removeMax(heap);
    }
}

int main(int argc, char *argv[]) {
    FILE *file;
    MaxHeap heap;

    char line[1000];

    heap.size = 0;

```

```

if (argc < 2) {
    printf("Error: no input file provided\n");
    return 1;
}

file = fopen(argv[1], "r");

if (file == NULL) {
    printf("Error: could not open file\n");
    return 1;
}

while (fgets(line, sizeof(line), file) != NULL) {

    /* remove newline character */
    line[strcspn(line, "\n")] = '\0';

    /* process EMAIL command */
    if (strncmp(line, "EMAIL ", 6) == 0) {
        Email email;
        char *firstComma;
        char *lastComma;

        firstComma = strchr(line + 6, ',');
        lastComma = strchr(line + 6, ';');

        if (firstComma != NULL && lastComma != NULL &&
            firstComma != lastComma) {

            /* get sender */
            *firstComma = '\0';
            strcpy(email.sender, line + 6);

            /* get subject */
            *lastComma = '\0';
            strcpy(email.subject, firstComma + 1);

```

```

        /* get date */
        strcpy(email.date, lastComma + 1);

        /* determine sender priority */
        email.priority = getPriority(email.sender);

        /* add email to MaxHeap */
        insertEmail(&heap, email);
    }
}

/* process NEXT command */
else if (strcmp(line, "NEXT") == 0) {
    displayNext(&heap);
}

/* process READ command */
else if (strcmp(line, "READ") == 0) {
    readEmail(&heap);
}

/* process COUNT command */
else if (strcmp(line, "COUNT") == 0) {
    printf("There are %d emails to read.\n\n", heap.size);
}
}

fclose(file);

return 0;
}

```

Claude

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

```

```

#define INITIAL_CAPACITY 16

#define MAX_LINE 512

typedef struct {

    char sender[32];    /* sender category: Boss, Subordinate, etc. */

    char subject[256];  /* subject line (may contain spaces) */

    char date[16];      /* MM-DD-YYYY, kept as given for display */

    int  priority;       /* category priority: Boss=5 ... OtherPerson=1 */

    long dateValue;      /* date encoded as YYYYMMDD for comparison */

} Email;

typedef struct {

    Email *data;

    int size;

    int capacity;

} MaxHeap;

/* ===== Max-Heap (array-based) ===== */

void heapInit(MaxHeap *h) {

    h->capacity = INITIAL_CAPACITY;

    h->size = 0;

    h->data = (Email *)malloc(sizeof(Email) * h->capacity);

    if (!h->data) {

        fprintf(stderr, "Memory allocation failed.\n");

        exit(1);

    }

}

void heapFree(MaxHeap *h) {

    free(h->data);

    h->data = NULL;

    h->size = 0;

    h->capacity = 0;

}

```

```

void heapResize(MaxHeap *h) {
    h->capacity *= 2;
    Email *newData = (Email *)realloc(h->data, sizeof(Email) * h->capacity);
    if (!newData) {
        fprintf(stderr, "Memory reallocation failed.\n");
        exit(1);
    }
    h->data = newData;
}

```

```

void swapEmails(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

```

/* Returns 1 if email a outranks (should be read before) email b. */

```

int hasHigherPriority(Email *a, Email *b) {
    if (a->priority != b->priority) {
        return a->priority > b->priority;
    }
    return a->dateValue > b->dateValue; /* newer date wins ties */
}

```

```

void heapifyUp(MaxHeap *h, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (hasHigherPriority(&h->data[index], &h->data[parent])) {
            swapEmails(&h->data[index], &h->data[parent]);
            index = parent;
        } else {
            break;
        }
    }
}

```

```

void heapifyDown(MaxHeap *h, int index) {

```

```

while (1) {
    int left = 2 * index + 1;
    int right = 2 * index + 2;
    int largest = index;
    if (left < h->size && hasHigherPriority(&h->data[left], &h->data[largest])) {
        largest = left;
    }
    if (right < h->size && hasHigherPriority(&h->data[right], &h->data[largest])) {
        largest = right;
    }
    if (largest == index) {
        break;
    }
    swapEmails(&h->data[index], &h->data[largest]);
    index = largest;
}
}

```

```

void heapInsert(MaxHeap *h, Email e) {

```

```

    if (h->size == h->capacity) {
        heapResize(h);
    }

```

```

    h->data[h->size] = e;
    heapifyUp(h, h->size);
    h->size++;
}

```

```

/* Removes and returns the highest-priority email. Caller must ensure size > 0. */

```

```

Email heapExtractMax(MaxHeap *h) {

```

```

    Email top = h->data[0];
    h->size--;
    h->data[0] = h->data[h->size];
    heapifyDown(h, 0);
    return top;
}

```

```

/* Returns a pointer to the highest-priority email without removing it,

```

```

* or NULL if the heap is empty. */

```

```

Email *heapPeek(MaxHeap *h) {
    if (h->size == 0) return NULL;
    return &h->data[0];
}

int heapSize(MaxHeap *h) {
    return h->size;
}

/* ===== Parsing ===== */

int categoryPriority(const char *category) {
    if (strcmp(category, "Boss") == 0)        return 5;
    if (strcmp(category, "Subordinate") == 0)  return 4;
    if (strcmp(category, "Peer") == 0)         return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0)  return 1;
    return 0; /* unrecognized category: lowest priority */
}

/* Converts MM-DD-YYYY into an integer YYYYMMDD for easy comparison. */

long dateToValue(const char *date) {
    int month, day, year;
    if (sscanf(date, "%d-%d-%d", &month, &day, &year) != 3) {
        return 0;
    }
    return (long)year * 10000L + (long)month * 100L + (long)day;
}

void trim(char *str) {
    int start = 0;
    while (isspace((unsigned char)str[start])) start++;
    if (start > 0) memmove(str, str + start, strlen(str) - start + 1);
    int len = (int)strlen(str);
    while (len > 0 && isspace((unsigned char)str[len - 1])) {
        str[len - 1] = '\0';
        len--;
    }
}

/* Parses "Category,Subject,Date" into an Email struct.

```

* Only the first two commas are treated as delimiters, so a subject
* containing no commas (as guaranteed by the spec) is captured whole. */

```
int parseEmailFields(char *line, Email *e) {  
    char *fields[3];  
    int count = 0;  
    char *token = line;  
    char *comma;  
    while (count < 3) {  
        comma = strchr(token, ',');  
        if (comma != NULL && count < 2) {  
            *comma = '\0';  
            fields[count++] = token;  
            token = comma + 1;  
        } else {  
            fields[count++] = token;  
            break;  
        }  
    }  
    if (count < 3) {  
        return 0; /* malformed line; skip */  
    }  
    trim(fields[0]);  
    trim(fields[1]);  
    trim(fields[2]);  
  
    strncpy(e->sender, fields[0], sizeof(e->sender) - 1);  
    e->sender[sizeof(e->sender) - 1] = '\0';  
    strncpy(e->subject, fields[1], sizeof(e->subject) - 1);  
    e->subject[sizeof(e->subject) - 1] = '\0';  
    strncpy(e->date, fields[2], sizeof(e->date) - 1);  
    e->date[sizeof(e->date) - 1] = '\0';  
    e->priority = categoryPriority(e->sender);  
    e->dateValue = dateToValue(e->date);  
    return 1;  
}
```

```

/* ===== Command handlers ===== */

void handleEmailCommand(MaxHeap *h, char *rest) {
    Email e;
    if (parseEmailFields(rest, &e)) {
        heapInsert(h, e);
    }
}

void handleNextCommand(MaxHeap *h) {
    Email *top = heapPeek(h);
    if (top == NULL) {
        printf("No emails to read.\n");
        return;
    }
    printf("Next email:\n");
    printf("Sender: %s\n", top->sender);
    printf("Subject: %s\n", top->subject);
    printf("Date: %s\n", top->date);
}

void handleReadCommand(MaxHeap *h) {
    if (heapSize(h) == 0) {
        return; /* nothing to read; produce no output */
    }
    heapExtractMax(h); /* discard: CEO has read/handled it */
}

void handleCountCommand(MaxHeap *h) {
    printf("There are %d emails to read.\n", heapSize(h));
}

/* ===== Main ===== */

int main(void) {
    MaxHeap heap;
    heapInit(&heap);
    char line[MAX_LINE];
    while (fgets(line, sizeof(line), stdin) != NULL) {
        trim(line);
        if (strlen(line) == 0) {

```

```

        continue;
    }
    if (strncmp(line, "EMAIL", 5) == 0 &&
        (line[5] == '\0' || isspace((unsigned char)line[5]))) {
        char *rest = line + 5;
        while (isspace((unsigned char)*rest)) rest++;
        handleEmailCommand(&heap, rest);
    } else if (strcmp(line, "NEXT") == 0) {
        handleNextCommand(&heap);
    } else if (strcmp(line, "READ") == 0) {
        handleReadCommand(&heap);
    } else if (strcmp(line, "COUNT") == 0) {
        handleCountCommand(&heap);
    }
    /* Any other/unknown line is silently ignored. */
}
heapFree(&heap);
return 0;
}

```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Correctness:

Claude:

Good: This program complies and runs as expected. It correctly implements a MaxHeap to read and store the emails. It handles all inputs

Bad: this program also assumes good input so for things not correctly input it would (most likely) silently fail as cause possible errors later on

Chatgpt:

Good: This program complies and runs as expected. It correctly implements a MaxHeap to read and store the emails. It handles all.

Bad: The program assumes good input and that the file is formatted correctly if there was an invalid format the program ignores it and moves on without an error. Also in the next email will only strip the `/r/n` and will not strip any potential spaces.

Execution time:

Claude:

Good: since this is a heap implementation this will have a big $O(\log n)$ which is pretty fast for insert and extract.

Bad: best big O speed with heap implementation.

Chatgpt:

Good: since this is a heap implementation this will have a big $O(\log n)$ which is pretty fast for insert and extract.

Bad: best big O speed with heap implementation

Space complexity:

Claude:

Good: this program uses a dynamic array based maxheap so the emails are stored in n array (space $O(n)$)

Bad:

Chatgpt:

Good: this program uses fixed list based maxheap so the emails are stored in n array: space $O(n)$

Bad: the program only stores enough space for 1000 emails but it reserves that no matter what which could end up just with unused occupied space.

Maintainability:

Claude:

Good: names are clear and functions are well organized and decently simple. Some edge cases are handled such as empty heap.

Bad: no comments only documented headers.

Chatgpt:

Good: the programs have been separated into different programs and have different purposes. Names are descriptive. Some edge cases are handled such as empty heap. This is easier and simpler to read than Claude but does a little less “fancy things” that might cause errors

Bad: A few more comments than Claude but lack of comments. There could be some better error handling rather than so ignoring for edge cases

f. Select one to use as the basis for this assignment and justify why you picked it.

I picked the Claude code because it wins on all the categories except time at which both programs are pretty equal. For correctness the Claude code handles a few errors better and is cleaner with issues. For space complexity it is the biggest winner because it does dynamic space instead of just setting 1000 for an array. For maintainability it wins just because I think the names the variables are a bit clearer and functions are organized a little better.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness: I think it needs to handle potential bad input a bit better. For example, making sure that things are as said as in instructions for the email format.

Execution time: I don't think there is much to improve the program runs at optimal $O(\log n)$

Space complexity: I think this implementation is done well with the dynamic spacing

Maintainability: I need to add more detailed line by line comments that are clearer.

3. Make the changes identified in 2g above.

I added comments that added clarity to the program additionally I added a new isValidEmail feature that ensures that the email input is correctly formatted

4. Compile and debug your program as described in the “C Basics” lecture.

5. Push both your source code, final cycle server executable file, and PDF of your

GenAI analysis to your GitHub repository https://github.com/CarterGary-17/Fall2026_EECS348_Assignment_2